

```
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2
```

```
===== 23.8/23.8 MB 25.2 MB/s eta 0:00:00  
===== 232.6/232.6 kB 24.0 MB/s eta 0:00:00
```

```
import faiss  
import numpy as np  
import torch  
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM  
from PyPDF2 import PdfReader  
from google.colab import files
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"  
print("Using device:", device)
```

```
Using device: cuda
```

```
enc_name = "sentence-transformers/all-MiniLM-L6-v2"  
  
enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)  
enc_model = AutoModel.from_pretrained(enc_name).to(device)  
enc_model.eval()
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%                               612/612 [00:00<00:00, 23.2kB/s]

tokenizer_config.json: 100%                       350/350 [00:00<00:00, 7.71kB/s]

vocab.txt: 232k/? [00:00<00:00, 3.85MB/s]

tokenizer.json: 466k/? [00:00<00:00, 5.49MB/s]

special_tokens_map.json: 100%                    112/112 [00:00<00:00, 2.64kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
model.safetensors: 100%                         90.9M/90.9M [00:01<00:00, 456MB/s]

Loading weights: 100%                           103/103 [00:00<00:00, 444.47it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 384, padding_idx=0)
    (position_embeddings): Embedding(512, 384)
    (token_type_embeddings): Embedding(2, 384)
    (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-5): 6 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=384, out_features=384, bias=True)
            (key): Linear(in_features=384, out_features=384, bias=True)
            (value): Linear(in_features=384, out_features=384, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=384, out_features=384, bias=True)
            (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=384, out_features=1536, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=1536, out_features=384, bias=True)
          (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=384, out_features=384, bias=True)
    (activation): Tanh()
  )
)

```

```

llm_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

llm_tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm_tokenizer.pad_token = llm_tokenizer.eos_token

llm_model = AutoModelForCausalLM.from_pretrained(
    llm_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    device_map="auto"
)

```

```
llm_model.eval()
```

```
config.json: 100% 608/608 [00:00<00:00, 8.02kB/s]

tokenizer_config.json: 1.29k/? [00:00<00:00, 17.0kB/s]

tokenizer.json: 1.84M/? [00:00<00:00, 13.9MB/s]

tokenizer.model: 100% 500k/500k [00:00<00:00, 2.53MB/s]

special_tokens_map.json: 100% 551/551 [00:00<00:00, 17.5kB/s]

`torch_dtype` is deprecated! Use `dtype` instead!

model.safetensors: 100% 2.20G/2.20G [00:26<00:00, 54.5MB/s]

Loading weights: 100% 201/201 [00:04<00:00, 47.44it/s, Materializing param=model.norm.weight]

generation_config.json: 100% 124/124 [00:00<00:00, 10.8kB/s]

LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 2048)
    (layers): ModuleList(
      (0-21): 22 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)
```

```
uploaded = files.upload()
pdf_path = list(uploaded.keys())[0]
```

Role of Mot...ld health.pdf
Role of Mother in Child health.pdf(application/pdf) - 162662 bytes, last modified: 3/17/2026 - 100% done
 Saving Role of Mother in Child health.pdf to Role of Mother in Child health.pdf

```
def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text += page_text + "\n"
    return text
```

```
def chunk_text(text, chunk_size=200, overlap=50):
    words = text.split()
    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap

    return chunks
```

```
def encode_chunks(chunks, batch_size=8):
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(chunks), batch_size):
```

```

        batch = chunks[i:i+batch_size]

        tokens = enc_tokenizer(
            batch,
            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to(device)

        outputs = enc_model(**tokens)
        embeddings = outputs.last_hidden_state.mean(dim=1)

        all_embeddings.append(
            embeddings.cpu().numpy().astype("float32")
        )

    return np.vstack(all_embeddings)

```

```

raw_text = extract_text_from_pdf(pdf_path)

doc_chunks = chunk_text(raw_text, chunk_size=200, overlap=50)
doc_embeddings = encode_chunks(doc_chunks)

faiss.normalize_L2(doc_embeddings)

dim = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dim)
index.add(doc_embeddings)

print("Total chunks indexed:", len(doc_chunks))

```

Total chunks indexed: 13

```

def retrieve_context(query, top_k=5):
    with torch.no_grad():
        tokens = enc_tokenizer(
            [query],
            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to(device)

        outputs = enc_model(**tokens)
        q_emb = outputs.last_hidden_state.mean(dim=1)
        q_emb = q_emb.cpu().numpy().astype("float32")

    faiss.normalize_L2(q_emb)

    scores, indices = index.search(q_emb, top_k)

    return [doc_chunks[i] for i in indices[0]]

```

```

def generate_rag_answer(question, top_k=5, max_new_tokens=256):
    contexts = retrieve_context(question, top_k)
    context_text = "\n\n".join(contexts)

    prompt = (
        "You are an assistant that answers ONLY using the provided context.\n\n"
        f"Context:\n{context_text}\n\n"
        f"Question: {question}\nAnswer:"
    )

    inputs = llm_tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True
    ).to(device)

    with torch.no_grad():
        output_ids = llm_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

```

```
output_text = llm_tokenizer.decode(
    output_ids[0],
    skip_special_tokens=True
)
```

```
print(output_text)
```

```
question = "What is the main contribution of this paper?"
```

```
answer = generate_rag_answer(question)
```

```
print("\nQuestion:", question)
```

```
print("\nAnswer:", answer)
```

```
Question: What is the main contribution of this paper?
```

```
Answer: The objectives of the Role of the Mother in the Child's Emotional Development are to reduce maternal and prenatal mortal
```

```
Question: What is the Role
```

```
while True:
```

```
    question = input("\nEnter your question (or type 'exit'): ")
```

```
    if question.lower() == "exit":
```